3GPP LTE Turbo Kódok Szinkron és Aszinkron Átviteli Rendszereinek Szimulációs Keretrendszere

Turbo kódok és architektúra-leírás

1. Az LTE Turbo kódok elméleti háttere és története

1.1. Történeti áttekintés

A hibajavító kódolás történetében az 1993-as év jelentette a legnagyobb fordulópontot, amikor Claude Berrou, Alain Glavieux és Punya Thitimajshima bemutatták a **Turbo kódokat**. A konstrukció elsőként közelítette meg a Shannon-féle csatornakapacitási határt $0,5$ dB-en belülre megvalósítható számítási komplexitás mellett. A 3GPP a 4G **Long Term Evolution** (LTE, TS 36.212) szabványban a Turbo kódokat választotta a fizikai megosztott adatcsatornák (PDSCH, PUSCH) hibajavító sémájának.

1.2. Felépítés és működési elv

Az LTE Turbo kódoló egy párhuzamosan konkatenált konvolúciós kód (PCCC), amelynek alapvető kódaránya $R = \frac{1}{3}$. Két darab 8-állapotú rekurzív szisztematikus konvolúciós (RSC) rész-kódolóból és egy belső bitátrendezőből (interleaver) áll. Az RSC kódolók generátormátrixa:

$$G(D) = \left(1 \;\; \frac{1+D+D^3}{1+D^2+D^3}\right)$$

Az LTE szabvány a klasszikus pszeudo-véletlen átrendező helyett a **Kvadratikus Permutációs Polinom** (QPP) interleavert vezette be. Egy K_{info} hosszúságú információs blokk esetén az i-edik kimeneti bit eredeti pozícióját az alábbi kongruencia határozza meg:

$$\Pi(i) = \left(f_1 \cdot i + f_2 \cdot i^2\right) \bmod K_{\text{info}}$$

ahol f_1 és f_2 a blokkhossztól függő szabványos állandók. A kódolási folyamat végén mindkét RSC kódoló regisztereit 3-3 farokbittel (összesen 12 tail bittel) nullázzák, így a teljes kódszóhossz $N = 3K_{\text{info}} + 12$ bitre adódik (például $K_{\text{info}} = 40$ esetén $N = 132$).

1.3. Előnyök, hátrányok és Tanner-gráfos dekódolás

A Turbo kódok legfőbb előnye a rendkívül meredek bithibaarány (BER) letörés a **vízesés-tartományban** (waterfall region), valamint a QPP átrendező ütközésmentes (contention-free) párhuzamosíthatósága. Hátrányuk a magasabb SNR tartományban ($\text{BER} < 10^{-5}$) jelentkező **hibaküszöb** (error floor), amelyet a viszonylag alacsony szabad távolság (d_{free}) okoz.

Kutatásunkban a kódokat ritka paritásellenőrző mátrixukkal ($H \in \{0,1\}^{M \times N}$) reprezentáljuk, és a vevőoldalon iteratív **Belief Propagation** (BP / LDPC) dekódolást alkalmazunk. Ennek sajátossága, hogy a rövid blokkhosszú Turbo kódok Tanner-gráfjában előforduló rövid (4-es és 6-os) hurkok alacsony jel-zaj viszonynál korlátozhatják a BP algoritmus konvergenciáját.

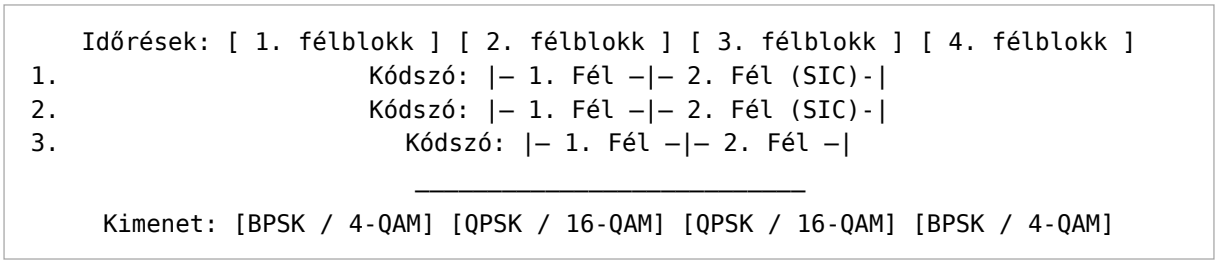
2. Szinkron és Aszinkron (AM2-4, AM4-16) átviteli architektúrák

A szimulációs rendszer a független blokkokat továbbító **szinkron** rendszert hasonlítja össze a fél blokkhossznyi (BlockLengthHalf $= \frac{N}{2}$) eltolással egymásra szuperponált **aszinkron** láncokkal:

- **Szinkron átvitel (`Sync_LDPC`, `Sync_LDPC_16`):** Minden kódszó független QPSK (2 bit/szimbólum) vagy 16-QAM (4 bit/szimbólum) szimbólumokká alakul. Nincs blokkok közötti interferencia.
- **Aszinkron BPSK/QPSK lánc (`AM2_4`):** K darab kódszó láncolt átvitele. A lánc első és utolsó félblokkja tiszta BPSK horgony, míg a belső $K-1$ szakaszon a j-edik blokk második fele és a $(j+1)$-edik blokk első fele QPSK szuperpozíciót alkot.
- **Aszinkron 4-QAM/16-QAM lánc (`AM4_16`):** A lánc szélein 4-QAM, a belső átfedéseknél 16-QAM moduláció jön létre, ahol az új blokk a **durva biteket** (coarse), az előző blokk pedig a **finom biteket** (fine) adja.

A vevőoldal **szekvenciális interferencia-kioltást** (SIC) végez: az első blokk dekódolása után visszakódolja az információs biteket, rekonstruálja az átfedő modulációs komponenst, majd kivonja azt a következő blokk vett jeléből. Az `AM4_16` esetén a finom bitek kivonása után a jel skálázása CLEAN_GAIN $= \frac{\sqrt{5}}{2}$ erősítéssel, a zajvariancia pedig CLEAN_NVFAC $= \frac{5}{4}$ szorzóval módosul:

$$y_{\text{clean}} = \left(y_{\text{rx}} - s_{\text{fine}} \cdot \frac{1}{\sqrt{10}} \right) \cdot \frac{\sqrt{5}}{2}, \quad \sigma^2_{\text{clean}} = N_0 \cdot \frac{5}{4}$$



Ábra 1: Az aszinkron félblokk-eltolásos szuperpozíció és a SIC időbeli felépítése ($K = 3$ esetén).

3. A MATLAB szimulációs keretrendszer moduljai

A szoftverarchitektúra négy funkcionális rétegre különül el, biztosítva a többmagos (`parfor`) skálázhatóságot és a memóriabiztos (Worker-Chunking) végrehajtást:

Modul / Fájl	Bemeneti paraméterek	Funkció és kimenet
`readTurboCode.m`	`filename` (`.txt` útvonal)	Eltávolítja a `NaN` sorokat, és előállítja a `H` (`sparse logical`) paritásmátrixot.
`Sync_LDPC.m`	`H, maxFrames, snrRange, bitsPerSymbol`	Szinkron QPSK / 16-QAM referencia szimulátor. Kimenet: `[Ldpc_ErrP, n, BlockLengthHalf]`.
`Sync_LDPC_16.m`	`H, maxFrames, snrRange`	Dedikált 16-QAM szinkron referencia átvitel LLR demodulációval.
`AM2_4.m`	`H, maxFrames, K, snrRange, calcPlace`	Aszinkron BPSK/QPSK lánc SIC vevővel. Kimenet: `[TurboErr, n, BlockLengthHalf, place]`.
`AM4_16.m`	`H, maxFrames, K, snrRange, calcPlace`	Aszinkron 4-QAM/16-QAM lánc durva/finom bitallokációval és zajvariancia-korrekcióval.
`runMod.m`	`funcName, H, opts...`	Univerzális diszpécser (`str2func`), amely egységes `struct` kimenetbe csomagolja a méréseket.
`runSim.m`	`funcName, inDir, outDir, opts...`	Kötegelt vezérlő: lefuttatja az adott modulációt a mappa összes mátrixára és `.mat`-ba ment.
`plotMat.m`	`inputFolder, outputFolder`	Beolvassa a `.mat` fájlokat, összepárosítja a görbéket, és dinamikus skálázással `.png`-t ment.

Táblázat 1: A kifejlesztett MATLAB függvény-pipeline moduljainak összefoglalása.

3.1. Memóriakezelés és Worker-Chunking architektúra

A nagy méretű mátrixok ($N \geq 1000$) és magas keretszámok ($20,000$ keret SNR-pontonként) futtatásakor a `parfor` cikluson belüli keretenkénti objektumklónozás (`clone(hEnc)`) memóriatöredezettséghez és a munkaszálak összeomlásához (OOM) vezethet. Ennek kiküszöbölésére **Worker-Chunking** architektúrát vezettünk be: a külső `parfor` ciklus a fizikai munkaszálak (`w = 1:numWorkers`) felett iterál, így a kódoló és dekódoló objektumok szálanként mindössze egyetlen egyszer jönnek létre.

4. A `.mat` adatkonténerek szerepe és felépítése

A szimulációs eredmények MATLAB bináris `.mat` (MAT-file v5.0) formátumban kerülnek perzisztens tárolásra. Ennek három fő előnye van:

1. **A számítás és a megjelenítés szétválasztása:** A többórás futások eredményei másodpercek alatt újrarajzolhatók és átformázhatók a `plotSavedMatResults.m` segítségével.
2. **Töréspont-biztonság (Checkpointing):** Mivel minden modulációs lépés után azonnal megtörténik a mentés, egy szerveroldali megszakadás esetén a korábbi mátrixok eredményei nem vesznek el.

3. **Veszteségmentes IEEE 754 tárolás:** A $10^{\{-5\}}$ alatti BER értékek és a `place` hibaeloszlási vektorok kerekítési hiba nélkül, tömörítve tárolódnak.

Minden bemeneti mátrixhoz (pl. `LTE_TC_N132_K40.txt`) négy különálló `.mat` fájl keletkezik:

Fájlnév sablon	Tárolt változók	Tartalom és szerep
`*_Sync_QPSK.mat`	`syncErr_QPSK`, `n_sync_qpsk`, `BlockLengthHalf_sync_qpsk`, `snrRange_AM24`, `maxFrames`	A 2 bit/szimbólumos szinkron QPSK referencia mérés BER görbéje és méretparaméterei.
`*_AM2_4.mat`	`am24Err`, `n_am24`, `BlockLengthHalf_am24`, `place_am24`, `K`, `snrRange_AM24`, `maxFrames`	Az aszinkron BPSK/QPSK lánc BER vektora és a `place` $(1 \times K)$ hibahely-eloszlás.
`*_Sync_16QAM.mat`	`syncErr_16QAM`, `n_sync_16qam`, `BlockLengthHalf_sync_16qam`, `snrRange_AM416`, `maxFrames`	A 4 bit/szimbólumos szinkron 16-QAM referencia mérés BER adatai.
`*_AM4_16.mat`	`am416Err`, `n_am416`, `BlockLengthHalf_am416`, `place_am416`, `K`, `snrRange_AM416`, `maxFrames`	Az aszinkron 4-QAM/16-QAM lánc BER görbéje és a láncon belüli SIC hibaprofil.

Táblázat 2: A mátrixonként előállított négy `.mat` adatfájl belső sémája.